Cs 201 Homework 2

Name: Eralp Yiğit Boz
Student No: 22403188
Section: 2

Task 1

1. addStudentWizard()
   a. Complexity: O(N)
   b. Justification: Validating if the student wizard already exists and copying the elements one by one both take N steps. The mathematical equation is T(N) = 2N + c, which simplifies to O(N).

2. removeStudentWizard()
   a. Complexity: O(N)
   b. Justification: Finding the target wizard takes N steps and copying the remaining elements takes N - 1 steps. The mathematical equation is T(N) = 2N - 1, which simplifies to O(N).

3. brewPotion()
   a. O(N + P)
   b. Justification: Finding the wizard index and checking for the potion takes N + P steps, and copying the existing potions takes P steps. The mathematical equation is T(N, P) = N + 2P + c, which simplifies to O(N + P).

4. discardPotion()
   a. Complexity: O(N + P)
   b. Justification: Locating the wizard and the potion takes N + P steps, and shifting the remaining potions takes P - 1 steps. The mathematical equation is T(N, P) = N + 2P - 1, which simplifies to O(N + P).

5. transferPotion()
   a. Complexity: O(N + P)
   b. Justification: Finding both wizards and the potion takes 2N + 2P steps, and the array reallocations take 2P - 1 steps. The mathematical equation is T(N, P) = 2N + 4P - 1, which simplifies to O(N + P).

6. showAllStudentWizards()
   a. Complexity: O(N log N + N * P)
   b. Justification: Sorting the wizard pointers via merge sort takes N log N steps, and calculating total strengths requires N * P steps. The mathematical equation is T(N, P) = N + N log N + (N * P), which simplifies to O(N log N + N * P).
7. showStudentWizard()
   a. Complexity: O(N + P)

b. Justification: Finding the target wizard takes N steps, and calculating the total strength and printing the potions take P steps each. The mathematical equation is T(N, P) = N + 2P, which simplifies to O(N + P).

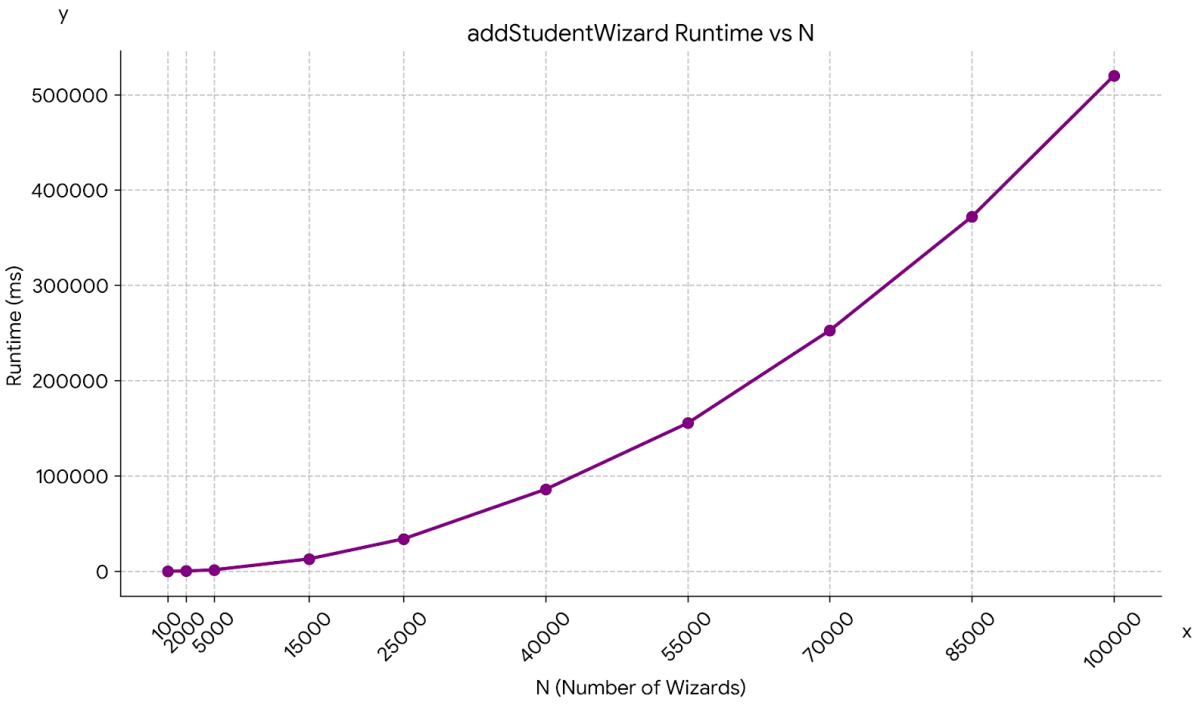
8. showPotion()
   a. Complexity: O(N * P + N log N)
   b. Justification: Searching all potions and printing the results takes 2(N * P) steps, while sorting the found owners takes worst-case N log N steps. The mathematical equation is T(N, P) = 3(N * P) + N log N, which simplifies to O(N * P + N log N).

Task 2

N (Wizard)	addStudentWizard (ms)	removeStudentWizard (ms)	showAllStudentWizards (ms)	showStudentWizard (ms)	Total Time (ms)
100	0.709	0.012	0.186	0.008	0.915
2000	216.399	0.154	3.549	0.018	220.120
5000	1467.170	0.409	9.043	0.066	1476.688
15000	12893.000	1.384	27.569	0.226	12922.179
25000	33966.600	1.915	46.149	0.211	34014.875
40000	85983.500	3.417	73.874	0.449	86061.240
55000	155426.000	5.245	103.080	0.689	155535.014

N (Wizard)	addStudentWizard (ms)	removeStudentWizard (ms)	showAllStudentWizards (ms)	showStudentWizard (ms)	Total Time (ms)
70000	252685.000	5.691	130.874	0.940	252822.505
85000	371954.000	6.994	159.283	1.178	372121.455
100000	519862.000	8.878	188.514	1.430	520060.822

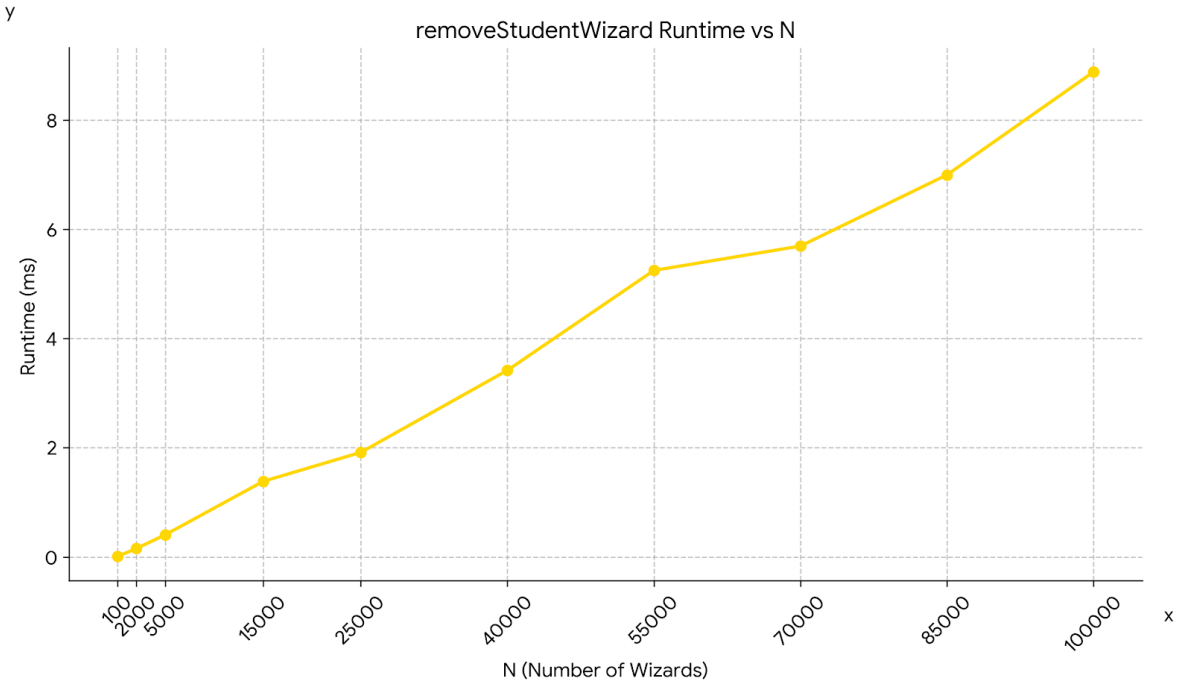
1) addStudentWizard



Explanation: As we did in task 1, the equation for addStudentWizard is T(N) = 2N + c, which simplifies to O(N). However, our test measures the sequential addition of many wizards from the beginning within a loop. Since this is a cumulative total, the measured time equation becomes N * (N + 1). As can be seen from the formula, the graph exhibits O(N^2) quadratic

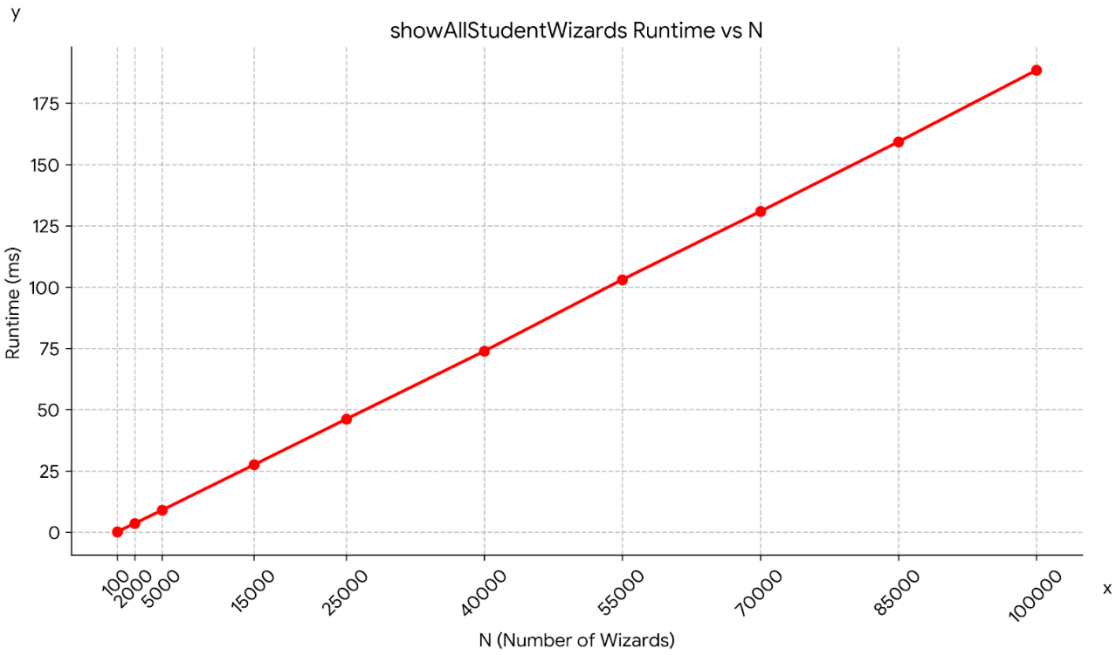
behavior and shows a parabolic curve reaching approximately 520,000 ms at a value of N=100,000.

2) removeStudentWizard



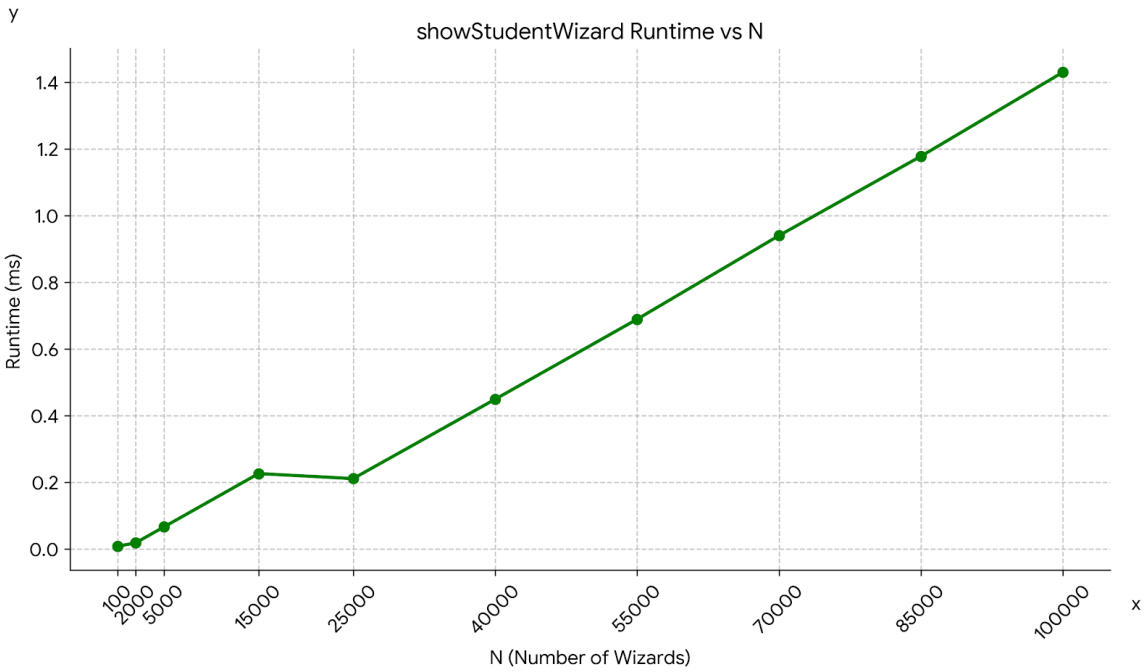
Explanation: The theoretical equation obtained in task 1 is T(N) = 2N - 1 which simplifies to O(N). The test algorithm measures a single removal from the middle of an N-element array — specifically the wizard at index N/2. Since only one removal is timed per N value, the graph directly reflects the O(N) cost of a single removeStudentWizard call as N grows, resulting in a linear graph consistent with the theoretical calculations. However, there is a slight anomaly between N=15,000 and N=25,000 where the runtime slightly decreases. The main reason for this is based on the working principle of the std::string::operator== method. In C++, string comparison first checks string lengths in O(1) time before comparing characters. When N is 15,000, the target wizard in the middle is named Wiz7500, which is 7 characters long. However, when N is 25,000, the target is Wiz12500, which is 8 characters long. Therefore, the == operator skips the first 10,000 wizards with length less than 8 with a size control of O(1) complexity, resulting in fewer detailed string comparisons and a slight reduction in runtime. Additionally, the slight plateau observed between N=55,000 and N=70,000 is likely caused by measurement noise from single-iteration timing, as small delays due to the server or internet connection are possible.

3) showAllStudentWizards



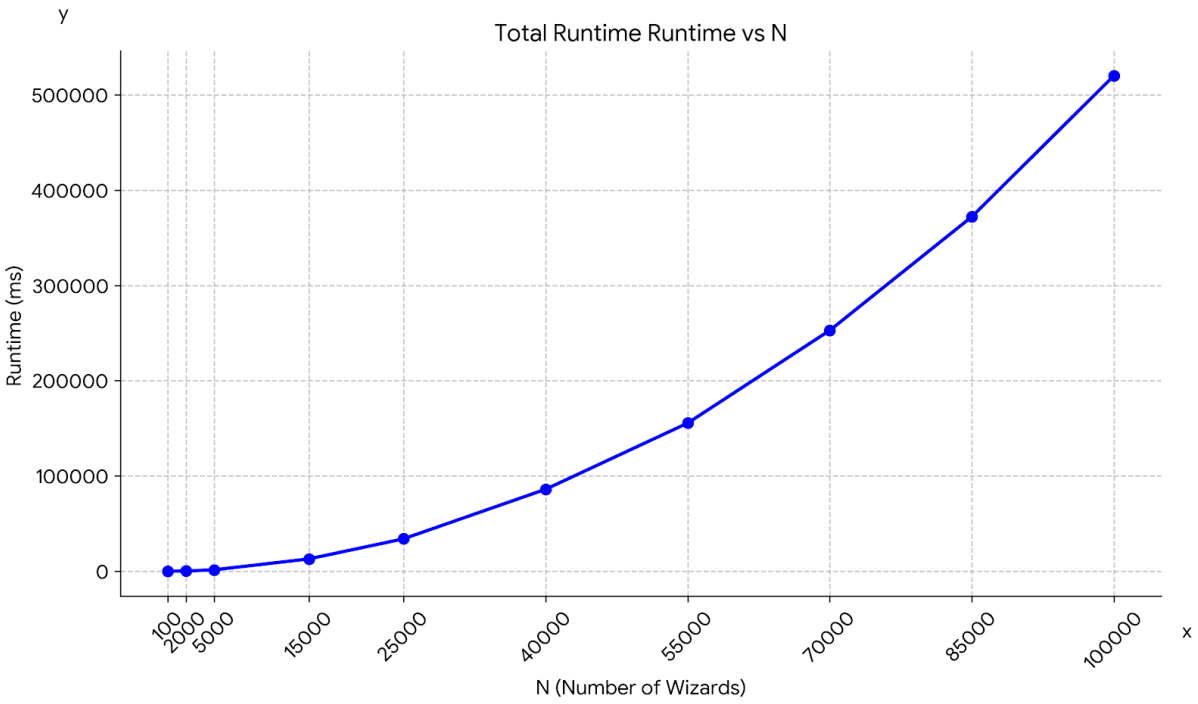
Explanation: In Task 1, I found the mathematical formula for this method as T(N,P) = N + N*log(N) + N*P, which simplifies to O(N*log(N) + N*P). Since the wizards in the test case have 0 potions, the speed of the method is determined not by the N*P part, but by the N*log(N) part from the merge sort algorithm. The graph obtained from the test case confirms the theoretical calculations. The graph resembles a linear graph, but as the value of N increases, the multiplier from log(N) slightly increases the growth rate of the graph.

4) showStudentWizard



Explanation: The theoretical time complexity derived in task 1 for finding and showing a specific wizard is T(N, P) = N + 2P, which simplifies to O(N). The graph obtained from the test case generally shows a linear trend, consistent with the theoretical calculations. A slight drop is observed between N=15,000 and N=25,000, which is caused by the std::string::operator== length-check behavior explained in detail in the removeStudentWizard section above.

5) Total Runtime



Explanation: Comparing the execution times of all operations for different N values clearly shows that the graph is quite similar to the addStudentWizard graph. Since the dynamic array reallocation and element-by-element deep copying operations performed in the addStudentWizard operation take much longer than other operations, the total time graph also resembles an O(n^2) graph, similar to the addStudentWizard graph.

Task 3

My LLM recommended these 6 things:

1) The LLM mentioned that creating a new array of size N+1 each time a new wizard is added is very costly for successive wizard additions. It suggested two solutions. The first was to completely change the architecture to a Custom Linked List to avoid contiguous memory reallocation entirely. The second was to keep the number of students and the array size separate, creating a new array twice the size of the existing one when it's full, and not creating a new array until the twice-sized array is full.

   My comment: The first solution it suggested didn't make sense to implement because our assignment needed to be about arrays, and dynamic arrays provide significantly better CPU cache locality compared to scattered linked list nodes. However, its second

suggestion, doubling the array size, is correct because it reduces unnecessary copying operations.

2) The LLM found it cumbersome to either create another array of size-1 or keep the array size fixed and shift all elements one position to the left during each delete operation in the existing array. Instead, it proposed keeping the array size fixed, swapping the element to be deleted with the last element, and deleting the swapped element after the swap operation.

   My comment: I agree with the LLM on this point as well. Its proposed solution saves us from having to copy or move the array repeatedly.

3) The LLM stated that passing strings by value in functions that take strings, such as addStudentWizard(const string name, const string house), causes unnecessary copying. Instead, it mentioned that passing strings by constant reference would reduce the strain on the system in large operations.

   My comment: I agree with the LLM's suggestion as well, since it reduces unnecessary copying operations.

4) The LLM noted that my current code uses linear search to find the wizard's index, which has O(N) complexity, and it told me I could use binary search instead, which has O(log(N)) complexity.

   My comment: I disagree with the LLM's suggestion. While using binary search speeds up the search process, it requires us to keep the array sorted. Therefore, each time we add a wizard, we need to perform O(N) complexity shifting in the array to place the new wizard in the correct position. Since maintaining a strictly sorted array is not a functional requirement of the system, the execution overhead of shifting elements during every insertion heavily outweighs the benefits of faster lookups.

5) My LLM indicated that creating two separate arrays (left and right) each time I used the `merge sort` method for sorting resulted in thousands of expensive heap allocations and releases during the sorting process. It suggested pre-allocating a single temporary workspace array in showAllStudentWizards and passing it down through the recursive calls to eliminate mid-sort memory allocations.

   My comment: I agree with the LLM. Instead of recursively creating new arrays repeatedly, performing operations on a single, predefined array will improve performance.

6) The LLM pointed out that the showPotion method unnecessarily performs two searches. The first is to count the matching wizards, and the second is to place the suitable wizards into the array. Instead, it suggested creating a small array initially and placing the matching wizards there in a single loop, then doubling the size of the array once it's full.

   My comment: I completely agree. Scanning the array twice is extremely inefficient. Solving the problem with a single loop significantly speeds up the program.

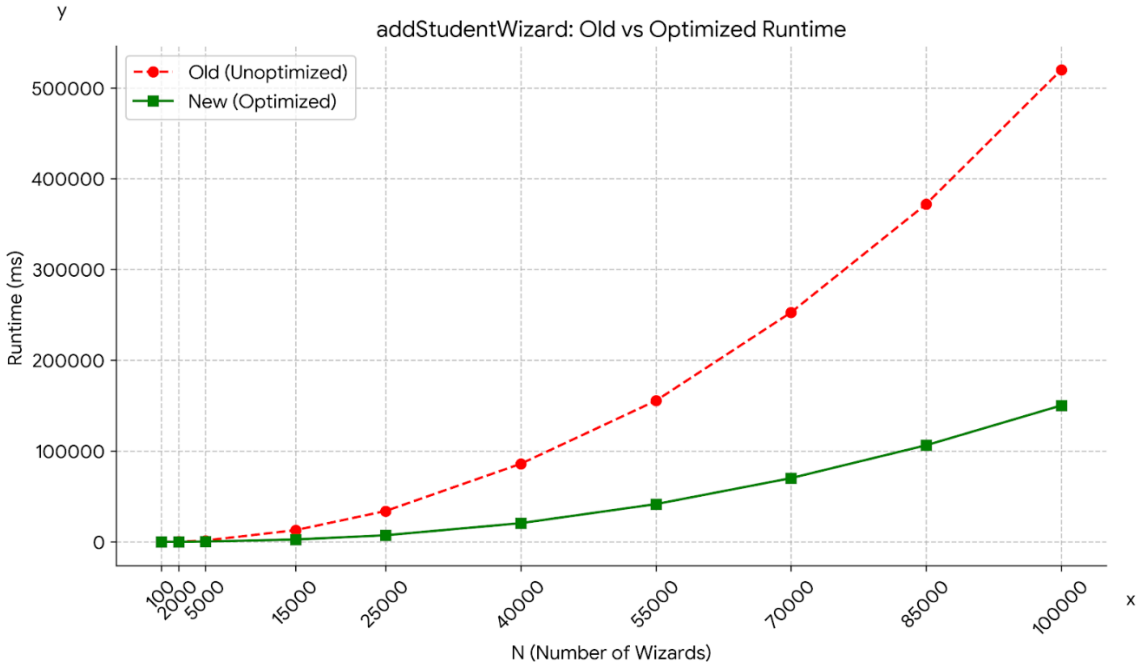
Task 4

In this task, I made the change suggested by the LLM in Task 3, along with the two mandatory changes.

N (Wizard)	**addStudentWizard (ms)**	**showAllStudentWizards (ms)**	**showStudentWizard (ms)**	**removeStudentWizard (ms)**	**Total Time (ms)**
100	0.387	0.190	0.007	0.005	0.589
2000	50.235	3.704	0.019	0.018	53.976
5000	349.232	9.360	0.072	0.069	358.733
15000	2662.930	28.693	0.243	0.233	2692.100
25000	7462.020	48.054	0.232	0.225	7510.530
40000	20881.400	77.326	0.487	0.474	20959.700
55000	41714.400	106.675	0.741	0.725	41822.500

N (Wizard)	addStudentWizard (ms)	showAllStudentWizards (ms)	showStudentWizard (ms)	removeStudentWizard (ms)	Total Time (ms)
70000	70051.900	136.898	0.998	0.978	70190.800
85000	105806.000	165.853	1.259	1.229	105974.000
100000	149050.000	195.362	1.528	1.477	149248.000

1) Dynamic Array Resizing



Instead of creating a new array each time as requested in the assignment, I designed a system that doubles the size of the array when it's full.

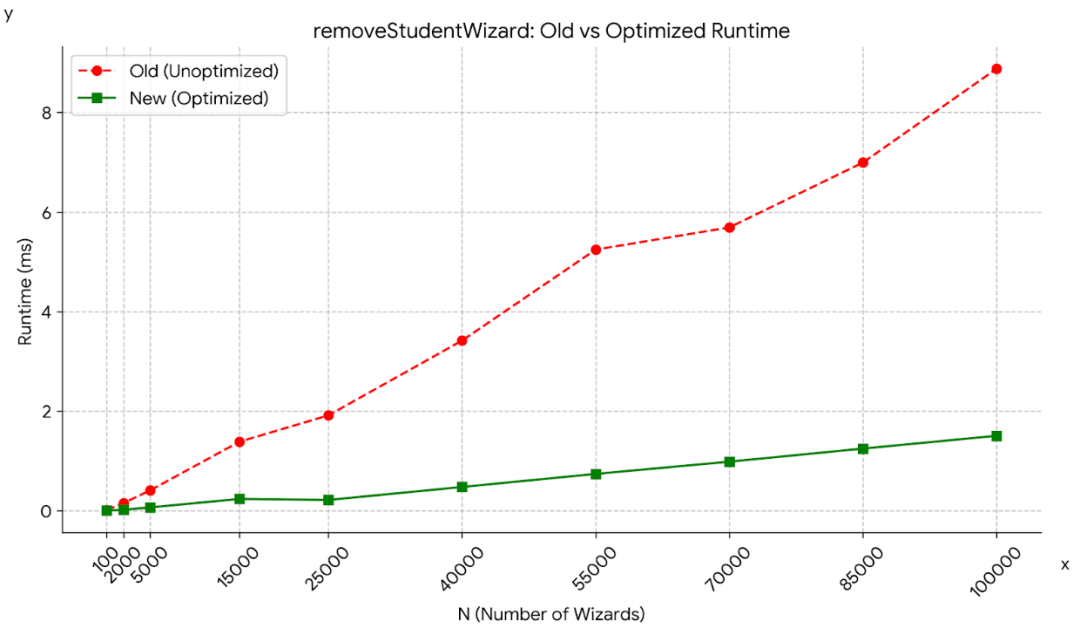
Amortized Analysis Task: Whenever the array reaches its full capacity (let's say N), we create an array twice the size of the original array and copy N elements into it. The number of elements copied at each capacity doubling step is 1, 2, 4, 8, ..., N. If we add N elements, the approximate formula becomes: $1 + 2 + 4 + 8 + \ldots + N/2 + N \approx 2N$. If

the total copy cost of 2N is divided equally among N operations, the average cost per insertion is 2N/N, which is 2. This corresponds to an amortized complexity of O(1).

New Complexity of addStudentWizard: Insertion (copying) step: amortized O(1); Full function: O(N) due to duplicate check

Explanation: In the old version of addStudentWizard, wizard scanning and copying operations were performed with O(N) complexity. In the new version, we reduced the O(N) copying operation to amortized O(1) complexity, resulting in a significant gain in code efficiency. However, due to the O(N) complexity from the wizard scanning, which prevents the existence of a wizard with the same name, the function efficiency remains at O(N) complexity even though efficiency has increased. Furthermore, since we call the O(N) function N times in the test loop, the cumulative cost becomes $O(N^2)$, which explains the parabolic shape. However, compared to the old version where both scanning and copying were O(N), the new version eliminates the copying overhead. This is reflected in the graph where N=100,000 dropped from ~520,000 ms to ~149,000 ms.

2) Swap-with-Last Removal

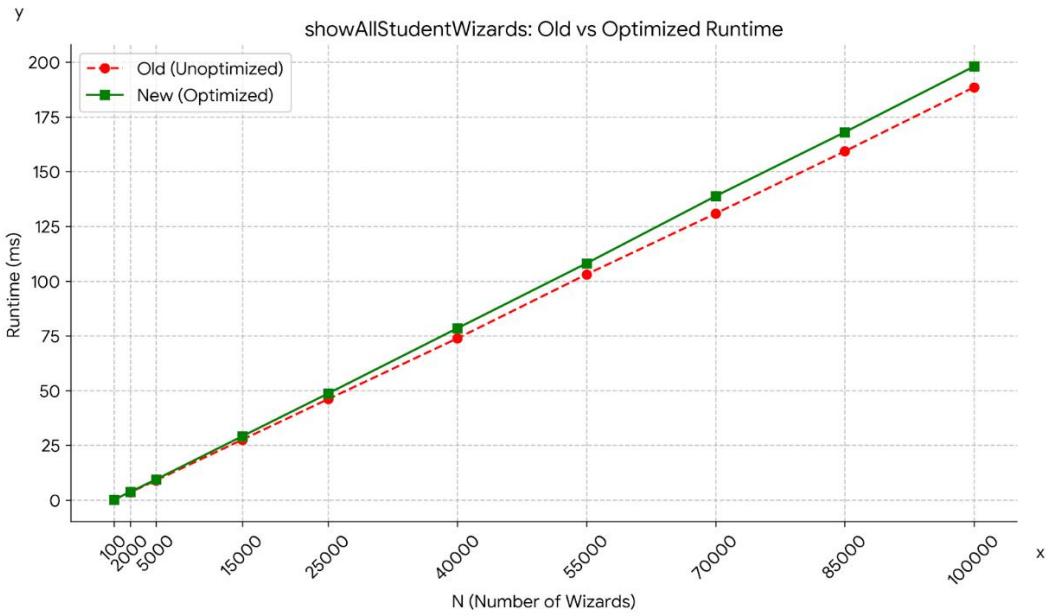


Complexity: O(N)

Explanation: In the old version of the removeStudentWizard method, after finding the person to be deleted with O(N) complexity, all elements on the right were shifted one step to the left to fill the gap in the array. This shifting operation took N/2 steps on average and was highly cumbersome for the processor. With the change I made, the new code first replaces the target element with the last element in the array, then decrements wizardCount with O(1) complexity instead of the N/2 shifting operation. Additionally, a null-pointer check after deletion and an edge case guard for when the target is already the last element were added to ensure memory safety. As a result, the mathematical equation becomes T(N) = N + 1. Although the function is still O(N)

complexity due to the linear search required to find the target wizard, the actual execution time has gained massive optimization as confirmed by the benchmark graph, where a single removal from an N-element array dropped from ~8.878 ms to ~1.477 ms at N=100,000.

3) LLM Suggested Optimization



For this part of the assignment, as suggested by LLM, I've modified the mergeSort algorithm so that the process of creating a new array each time can be handled with a single array.

Complexity: O(N*log(N))

Explanation: I wrote a new mergeSort that uses a single array, replacing the old mergeSort's recursive array structure that created left and right arrays each time. I chose the showAllStudentWizards method to demonstrate the mergeSort's efficiency. Initially, I created an N-sized array inside the showAllStudentWizards method and then passed this array as a parameter in all recursive calls. Although the method complexity remains O(N*log(N)) because the basic idea of mergeSort is divide and conquer, the efficiency of the code has increased due to the reduced array creation and deletion operations. The minimal difference in execution times in the graph is due to the fact that the data size of 100,000 elements is not large enough. For arrays with more than 100,000 elements, the efficiency difference will be noticeable.

Task 5

1) Prompt I used:

a. For Optimization (I also submitted my graphics and code files here): I'm working on my CS 201 assignment. Task 3 states that I need to send you my code and discuss any optimization issues with you. (Using STL libraries like vector, list, etc. is STRICTLY FORBIDDEN). The steps are as follows:

   1. Provide the LLM with your source code and the graphs generated in Task2.
   2. Ask the LLM to identify which functions are creating performance bottlenecks and open
   to improvement why and how.
   3. Document the LLM's reasoning and comment briefly on whether it makes sense.

b. For Optimization: What else can you suggest besides these?

c. For debugging: I've implemented the optimization you suggested for Merge Sort; are there any issues with my code?

d. For debugging: You're saying this might cause problems, and you're right if the code is improved. But is there a problem with the current version, or is this problem only likely to occur in future versions?

2) The LLM's suggestions for bottlenecks and optimizations: As I mentioned in Task 3, the LLM offered six different suggestions for my code.

a. Dynamic Array / Linked List: Instead of increasing the array size by 1 with each addition, it suggested doubling the array capacity (Capacity Doubling) or switching entirely to a Linked List architecture.

b. Swap-with-Last: Instead of shifting the other elements in the array O(N) when deleting an element, it recommended that I swap the element to be deleted with the element at the end of the array O(1).

c. Pass by Reference: To avoid unnecessary memory copying, it suggested passing string variables to the function by reference (const string&).

d. Binary Search: To achieve search speeds of O(log(N)), it instructed me to perform a Binary Search by always keeping the array sorted.

e. Merge Sort Workspace: To avoid recursive new/delete waste within Merge Sort, it recommended creating a single temporary workspace outside and passing it as a parameter.

f. showPotion Optimization: Instead of scanning the array twice to find target wizards in the showPotion method, it suggested completing the task in a single loop using capacity doubling logic.

3) Reflection on how helpful (or unhelpful) the LLM was in the debugging/optimization process:

   Overall, I can say that the LLM was highly effective for both optimizing my code and debugging. While some of the solutions it suggested had occurred to me initially, suggestions like writing the merge sort with a single array, which I also used in my report, were important suggestions that didn't immediately come to mind and could be useful in large datasets. However, this assignment reminded me again that one shouldn't rely solely on the LLM. While its suggestion to keep the array sorted and perform binary search initially seemed logical, after some thought, I realized it wasn't very efficient. This would revert the wizard insertion time complexity, which we successfully optimized to amortized O(1) with capacity doubling, **back to O(N)** due to the shifting operations required. Apart from these, I think the LLM was quite effective for testing my code for errors. In conclusion, the LLM was an excellent assistant in finding the root causes of performance problems and offering alternative data structures. However, applying the solutions most suitable for the system architecture was up to me.

References

Google. (2026). *Gemini 3.1 Pro* [Large language model]. https://gemini.google.com/